

UNIT-5

NoSQL Databases

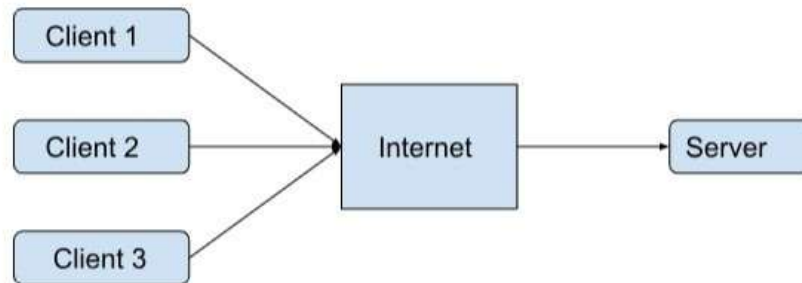
❖ **Distributed database system:**

- A distributed database is a database system that spans multiple computers or nodes that are connected by a network. Each node in a distributed database can store a portion of the data, and the entire database is made up of the sum of the data stored on each node.
- 1. **Homogeneous distributed databases:** In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used – all are the same at all sites. Hence, they're easy to manage.
- 2. **Heterogeneous Database:** In a heterogeneous distributed database, different sites can use different schema and software that can lead to problems in query processing and transactions. Also, a particular site might be completely unaware of the other sites. Different computers may use a different operating system, different database application. They may even use different data models for the database.

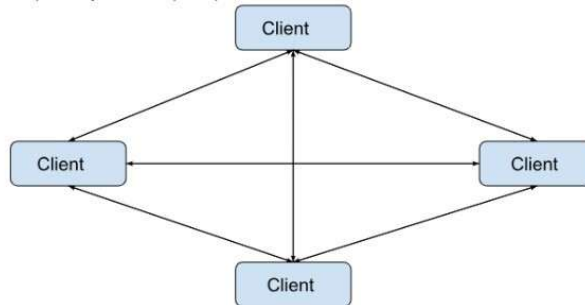
❖ **Architecture of DDS:** Distributed Database Systems can be structured in various ways, each offering unique benefits & challenges. The most common architectures include:

1. **Client-Server Model:** This is the most traditional form, where multiple client machines access a centralized server hosting the database. The server is in charge of maintaining data storage, controlling access, and organizing transactions. This architecture has several clients and servers connected. A client sends a query and the server which is available at the earliest would help solve it. This Architecture is simple to execute because of the centralized server system.

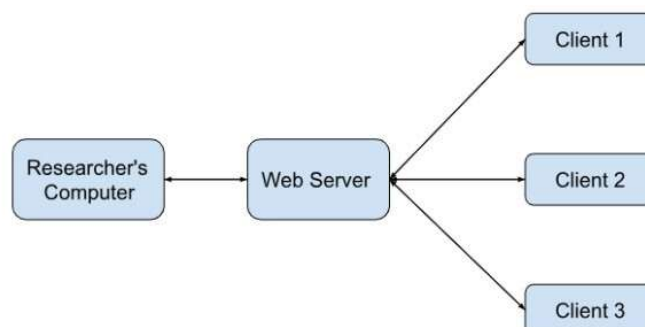
Database management systems



2. **Peer-to-Peer Architecture:** Each node in the distributed database system may function as both a client and a server in a peer-to-peer architecture. Each node is linked to the others and works together to process and store data.

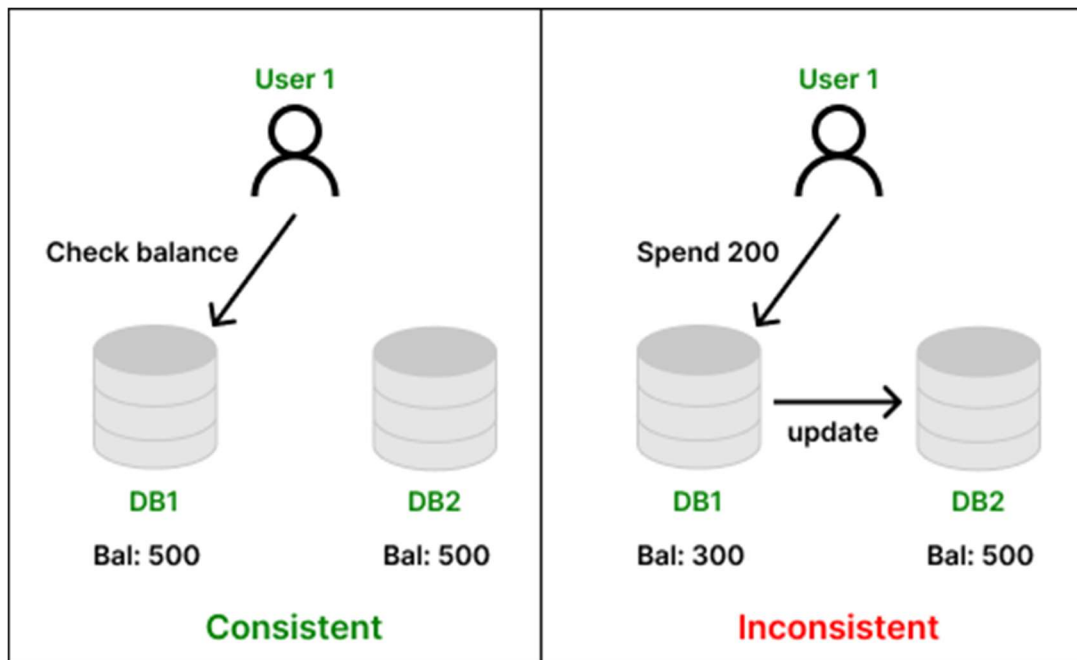


3. **Federated Architecture:** A type of database management system in which multiple autonomous, heterogeneous databases are integrated into a unified system while retaining their local control and independence. It offers a uniform interface for navigating and exploring distributed data. In the federated design, each site maintains a separate, independent database, while the virtual database manager internally distributes requests.

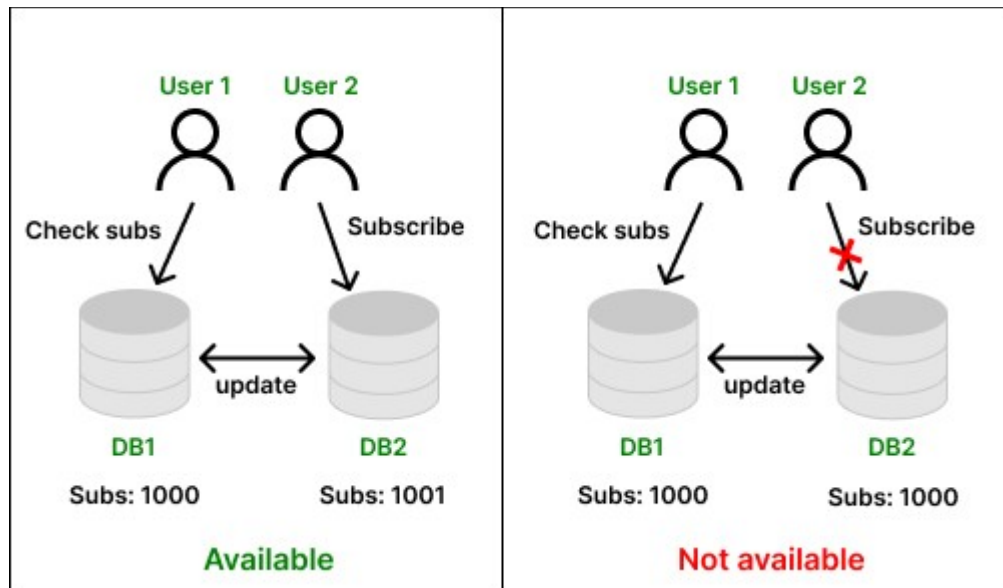


❖ **CAP theorem:** CAP Theorem, also known as **Brewer's Theorem**, states that in a **distributed database system**, it is **impossible** to simultaneously guarantee all three of the following properties.

1. **Consistency:** Consistency means that all the nodes (databases) inside a network will have the same copies of a replicated data item visible for various transactions. It guarantees that every node in a distributed cluster returns the same, most recent, and successful write. **Example:** If a user updates data (e.g., a bank balance), all nodes must reflect that updated data immediately.



2. **Availability:** Availability means that each read or write request for a data item will either be processed successfully or will receive a message that the operation cannot be completed. Every non-failing node returns a response for all the read and write requests in a reasonable amount of time. **For example**, user A is a content creator having 1000 other users subscribed to his channel. Another user B who is far away from user A tries to subscribe to user A's channel. Since the distance between both users are huge, they are connected to different database node of the social media network. If the distributed system follows the principle of availability, user B must be able to subscribe to user A's channel.



3. **Partition tolerance:** Partition tolerance means that the system can continue operating even if the network connecting the nodes has a fault that results in two or more partitions, where the nodes in each partition can only communicate among each other. That means, the system continues to function and upholds its consistency guarantees in spite of network partitions. **Example:** If your laptop loses internet connection while editing a Google Doc, you can keep working on it offline. Your changes are saved locally on your device. When the internet is restored, Google Docs syncs your edits to the server. Partition Tolerance: The system works even when disconnected and recovers seamlessly.

❖ **Big data:** Big Data refers to a collection of a very **large and complicated** set of data for which it becomes difficult to process using traditional or manual database management tools. The size of data **grows exponentially** and is generally in terabytes or more.

- **Types of data:**

1. **Structured data:** Any data in a fixed format is known as structured data. It can only be accessed, stored, or processed in a particular format. This type of data is stored in the form of tables with rows and columns. Any Excel file or SQL file is an example of structured data.
2. **Unstructured data:** Unstructured data do not have a fixed format. These are stored in an unknown format. It has no fixed schema or organization. Such type of data is known as unstructured data. An example of unstructured data is a web page with text, images, videos, etc.
3. **Semi-structured:** Semi-structured data is the combination of structured as well as unstructured forms of data. It does not contain any table to show relations; it contains tags or other markers to show hierarchy. JSON files, XML files, and CSV files (Comma-separated files) are semi-structured data examples. The e-mails we send or receive are also an example of semi-structured data.

❖ **NOSQL database:**

- NoSQL is a type of database management system (DBMS) that is designed to handle and store large volumes of unstructured and semi-structured data.
- NoSQL databases (AKA "not only SQL") store data differently than relational tables. NoSQL databases come in a variety of types based on their data model.
- The main types are document, key-value, wide-column, and graph. They provide flexible schemas and scale easily with large amounts of big data and high user loads.

- **Need/uses of NOSQL over RDBMS:**

1. When the amount of data grows too large for traditional relational databases, NoSQL provides a scalable solution. **Example:** Websites like **Facebook** and **Twitter** need to store massive amounts of user data, including posts, images, and comments. Traditional relational databases can't scale as easily.
2. NoSQL databases allow storing **unstructured or semi-structured data** like images, videos, or social media posts. They don't require a predefined schema, making it easier to handle dynamic data. **Example: MongoDB**, a NoSQL database, stores data in JSON-like documents, allowing users to add new fields to documents without affecting the overall structure.
3. NoSQL databases are optimized for fast read and write operations. This makes them ideal for applications that require high performance and low-latency access to data.
4. NoSQL databases can be scaled **horizontally**, meaning you can add more servers to handle more data and traffic, rather than just upgrading the existing hardware.
5. Some applications require the ability to handle different types of data (e.g., documents, key-value pairs, graphs). NoSQL databases can store and manage these various data types more efficiently than traditional relational databases.

- **Uses of NoSQL in Social Networking:**

1. **User Data Storage:** NoSQL databases store large volumes of user data, including profiles, posts, and media content in an efficient and flexible manner.
2. **Real-Time Analytics:** Social media platforms use NoSQL databases for real-time data processing, enabling features like live feeds, notifications, and instant messaging.
3. **Content Management:** NoSQL databases store and manage diverse content types, such as text, images, videos, and comments, which are common in social networking applications.

4. **Recommendation Engines:** NoSQL can handle large-scale user behaviour data to build recommendation algorithms, suggesting friends, posts, or advertisements.
5. **Scalable Infrastructure:** NoSQL enables social networking platforms to scale horizontally as user numbers grow, ensuring continued performance even under heavy loads.

Features

- 1) The NoSQL does not follow any relational model.
- 2) It is either schema free or have relaxed schema. That means it does not require specific definition of schema.
- 3) Multiple NoSQL databases can be executed in distributed fashion.
- 4) It can process both unstructured and semi-structured data.
- 5) The NoSQL have higher scalability.
- 6) It is cost effective.
- 7) It supports the data in the form of key-value pair, wide columns and graphs.

❖ NOSQL types:

1. **Key-value store:** A key-value store is a nonrelational database. The simplest form of a NoSQL database is a key-value store. Every data element in the database is stored in key-value pairs. The data can be retrieved by using a unique key allotted to each element in the database. The keys are unique but values can be simple data types like strings, JSON or binary objects. It is designed in such a way to handle lots of data and heavy load. **E.g.** Dynamo, Riak, Redis are some databases uses key-value store.

- For example -

```
{Customer:  
  [  
    {"id":1, "name":"Ankita"},  
    {"id":2, "name":"Kavita"}  
  ]  
}
```

- Here id, name are the keys and 1,2, "Ankita", "Prajakta" are the values corresponding to those keys.
- Key-value stores help the developer to store schema-less data . They work best for Shopping Cart Contents.

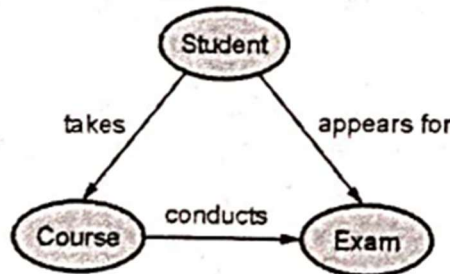
2. **Document store:** The document-based database is a nonrelational database. Instead of storing the data in rows and columns (tables), it uses the documents to store the data in the database. A document database stores data in JSON, BSON, or XML documents. MongoDB and CouchDB are two popular documents oriented NOSQL database.

So in the document data model, each document has a key-value pair below is an example for the same.

```
{  
  "Name" : "Yashodhra",  
  "Address" : "Near Patel Nagar",  
  "Email" : "yahoo123@yahoo.com",  
  "Contact" : "12345"  
}
```

3. **Graph:** Graph-based databases focus on the relationship between the elements. It stores the data in the form of nodes in the database. The connections between the nodes are called links or relationships. The nodes of graph are entities itself like people, students etc. The links are relationships. Updating data is very easy, as adding a new node or edge to a graph database is a straightforward task that does not require significant schema changes.

- For example -



- Graph base database is mostly used for social networks, logistics, spatial data. The graph databases are - Neo4J, Infinite Graph, OrientDB.

4. **Wide column stores:** A column-oriented database is a non-relational database that stores the data in columns instead of rows. The columns are created for each row rather than having predefined by the table structure. In this model the number of columns is not fixed for each record. While relational databases require all rows to have the same columns, wide-column stores allow different rows to have different columns. The schema is not rigid.

- For example -

Row ID	Columns...		
1	Name	City	
	Ankita	Pune	
2	Name	City	email
	Kavita	Mumbai	kavita123@gmail.com

- The column store databases are widely used to manage data warehouses, business intelligence, HBase, Cassandra are examples of column based databases.

❖ BASE Properties of NOSQL:

1. Basically available:

- It means the system is guaranteed to be available in the event of failure. It refers to the fact that the database system should always be available to respond to user requests, even if it cannot guarantee immediate access to all data.
- **Example:** In a social media app, if one server goes down, the app may still allow users to post or comment, even though the data on all servers may not be synchronized immediately.

2. Soft state:

- In simple terms, the soft state property means that a database can change over time, even if no one is actively making changes. These changes can happen because of things like automatic updates, background processes, or the system syncing data across different servers.

- However, even though the state of the database is changing, the system is designed to handle these changes smoothly. It ensures that this doesn't cause problems like losing or corrupting data. Instead, the database will eventually become consistent and synchronized, even if it's temporarily in an inconsistent state during the process.
- **Example:** In an online shopping system, if a product's stock count is updated in one part of the system, it may not reflect across all nodes right away. But eventually, the system will sync all the nodes to have the correct stock count.

3. Eventual consistency:

- This property refers to the eventual consistency of data in the database, despite changes over time.
- In simple terms, this property means that the database might not be perfectly consistent right away, but over time, it will become consistent. So, even if data changes and takes time to spread across different parts of the system, eventually, all the data will match up and be the same everywhere. This is different from traditional databases, which always try to make sure that all data is consistent immediately.
- **Example:** In a distributed chat application, if a message is sent, it may not immediately appear on every device connected to the system. However, after a short period, the message will eventually appear on all devices.

- **Significance:**

1. BASE helps databases grow and handle more data and users by not requiring data to be instantly consistent across all parts. This is important when dealing with large amounts of information, like in big data.
2. The "Basically Available" part makes sure the system keeps working, even if some parts break down. This helps the system stay reliable.
3. By allowing data to sync over time instead of immediately, BASE lets the system work faster. It doesn't need to wait for all data to match up before doing something.
4. BASE allows databases to store data in different ways, including semi-structured or unstructured data, giving more freedom in how data is organized and used.

5. Even if the data isn't perfectly synced right away, the system still works smoothly for users, providing a good experience in platforms like social media or online stores, where availability matters more than instant consistency.

❖ How Soft State Relies on Eventual Consistency:

- Soft state allows the system to operate while it's still syncing data between nodes. It doesn't require all data to be perfectly consistent at all times. This makes it possible to process requests and continue operations even if data is out-of-sync for a short period.
- Eventual consistency ensures that this temporary inconsistency doesn't last forever. As the system continues to operate, it eventually becomes consistent as all nodes sync up.
- In short, **soft state** means the system can tolerate temporary inconsistencies, whereas **eventual consistency** ensures that those inconsistencies will be resolved over time, so the system will reach a consistent state.
- Together, these two properties allow distributed systems (like NoSQL databases) to remain highly available and scalable while still ensuring that, over time, all data across the system will become consistent.

❖ ACID vs BASE properties:

Criteria	ACID	BASE
1. Simplicity	Simple: ACID systems are straightforward because they ensure every transaction is processed and committed correctly.	Complex: BASE systems are more complicated due to the focus on availability and scalability rather than strict consistency.
2. Focus	Commits: ACID ensures that all transactions are fully committed, meaning the data is always accurate and consistent.	Best attempt: BASE focuses on trying to ensure availability and scalability, even if it means data isn't fully consistent right away.
3. Maintenance	High: ACID systems need regular checks and balancing to maintain data consistency and correctness.	Low: BASE systems need less maintenance because they handle temporary inconsistencies and don't require strict consistency at all times.
4. Consistency of Data	Strong: ACID guarantees that data is consistent across all systems immediately.	Weak/Loose: BASE allows data to be inconsistent temporarily, but it will eventually become consistent over time.

Database management systems

5. Concurrency Scheme	Nested transactions: ACID handles multiple transactions at the same time, ensuring no conflicts and preserving consistency.	Close to answer: BASE handles concurrency with a best-effort approach, meaning it prioritizes availability even if some data is temporarily inconsistent.
6. Scaling	Vertical scaling: ACID typically scales by adding more power (like RAM, CPU) to a single server.	Horizontal scaling: BASE scales by adding more servers or nodes to distribute the load, making it better for handling large amounts of data.
7. Implementation	Easy to implement: ACID is easier to implement, especially for smaller systems where data consistency is key.	Difficult to implement: BASE is harder to implement due to the complexity of managing distributed systems, data synchronization, and eventual consistency.
8. Upgrade	Harder to upgrade: Upgrading ACID systems can be challenging, as it requires careful planning to avoid breaking consistency during upgrades. ↓	Easy to upgrade: BASE systems are easier to upgrade because they are designed to handle changes and scale without disrupting the entire system.
9. Type of Database	Robust: ACID systems are considered robust because they ensure strong data integrity, making them ideal for applications requiring precision (e.g., banking).	Simple: BASE systems are simpler and focus on handling large amounts of data and ensuring high availability, even if consistency isn't always perfect.
10. Type of Code	Simple: The code for ACID systems is simpler because it deals with straightforward transactions and ensures data integrity.	Harder: The code for BASE systems is more complex due to the need to handle distributed systems, eventual consistency, and fault tolerance.
11. Time Required for Completion	Less time: ACID transactions are completed quickly because the system waits for data to be fully committed before moving on.	More time: BASE systems may take longer because they don't require data to be fully synchronized at once and allow for eventual consistency.
12. Examples	Oracle, MySQL, SQL Server: These databases follow ACID principles and are used in applications where data consistency is critical. ↓	DynamoDB, Cassandra, CouchDB: These databases follow BASE principles and are used for large-scale applications that require high availability and scalability.

❖ **Difference between SQL and NOSQL:**

Index	SQL	NoSQL
1)	Databases are categorized as Relational Database Management System (RDBMS).	NoSQL databases are categorized as Non-relational or distributed database system.
2)	SQL databases have fixed or static or predefined schema.	NoSQL databases have dynamic schema.
3)	SQL databases display data in form of tables so it is known as table-based database.	NoSQL databases display data as collection of key-value pair, documents, graph databases or wide-column stores.
4)	SQL databases are vertically scalable.	NoSQL databases are horizontally scalable.
5)	SQL databases use a powerful language "Structured Query Language" to define and manipulate the data.	In NoSQL databases, collection of documents are used to query the data. It is also called unstructured query language. It varies from database to database.
6)	SQL databases are best suited for complex queries.	NoSQL databases are not so good for complex queries because these are not as powerful as SQL queries.
7)	SQL databases are not best suited for hierarchical data storage.	NoSQL databases are best suited for hierarchical data storage.
8)	MySQL, Oracle, Sqlite, PostgreSQL and MS-SQL etc. are the example of SQL database.	MongoDB, BigTable, Redis, RavenDB, Cassandra, Hbase, Neo4j, CouchDB etc. are the example of nosql database

❖ **Difference between NOSQL and RDBMS:**

NoSQL Database	Relational Database
NoSQL Database supports a very simple query language.	Relational Database supports a powerful query language.
NoSQL Database has no fixed schema.	Relational Database has a fixed schema.
NoSQL Database is only eventually consistent.	Relational Database follows acid properties. (Atomicity, Consistency, Isolation, and Durability)
NoSQL databases don't support transactions (support only simple transactions).	Relational Database supports transactions (also complex transactions with joins).
NoSQL Database is used to handle data coming in high velocity.	Relational Database is used to handle data coming in low velocity.
The NoSQL's data arrive from many locations.	Data in relational database arrive from one or few locations.
NoSQL database can manage structured, unstructured and semi-structured data.	Relational database manages only structured data.
NoSQL databases have no single point of failure.	Relational databases have a single point of failure with failover.
NoSQL databases can handle big data or data in a very high volume .	NoSQL databases are used to handle moderate volume of data.
NoSQL has decentralized structure.	Relational database has centralized structure.
NoSQL database gives both read and write scalability.	Relational database gives read scalability only.
NoSQL database is deployed in horizontal fashion.	Relation database is deployed in vertical fashion.

✚ **MongoDB:** MongoDB is a No SQL database. Mongo DB is a document-oriented database. It is an open-source product, developed and supported by a company named 10gen. MongoDB was designed to work with commodity servers. Now it is used by the company of all sizes, across all industry.

❖ **CRUD operations:** CRUD is an acronym for Create, Read, Update, and Delete. CRUD operations in MongoDB are used to manipulate data in databases.

➤ **Create:** The create or insert operations are used to insert or add new documents in the collection. If a collection does not exist, then it will create a new collection in the database.

You can perform, create operations using the following methods provided by the MongoDB:

Method	Description
<code>db.collection.insertOne()</code>	It is used to insert a single document in the collection.
<code>db.collection.insertMany()</code>	It is used to insert multiple documents in the collection.
<code>db.createCollection()</code>	It is used to create an empty collection.

➤ **Create Collection:** To create a new collection, you can use the "createCollection" method.

Syntax:

```
db.createCollection(name, options)
```

Example:

```
db.createCollection("users")
```

➤ **Insert one document:**

Syntax:

```
db.collectionName.insertOne()
```

Example:

```
db.users.insertOne({  
  name: "Angela",  
  age: 27,  
});
```

➤ **Insert many documents:**

Syntax:

```
db.collectionName.insertMany();
```

Example:

```
db.users.insertMany([  
  {  
    name: "Angela",  
    age: 27,  
  },  
  {  
    name: "Dwight",  
    age: 30,  
  },  
  {  
    name: "Jim",  
    age: 29,  
  }  
]);
```


- **Read:** In MongoDB, read operations are used to retrieve data from the database. There are two methods discussed below:
 - **Find ():** The find () method is used to retrieve data from a collection. It returns a cursor that can be iterated to access all the documents that match the specified query criteria.

Syntax:

```
db.collectionName.find(query, projection)
```

```
db.student.find()
```

Output:

```
[> db.student.find().pretty()
{
  "_id" : ObjectId("60227eaff19652db63812e8d"),
  "name" : "Akshay",
  "age" : 18
}
{
  "_id" : ObjectId("60227eaff19652db63812e8e"),
  "name" : "Bablu",
  "age" : 17,
  "score" : {
    "math" : 230,
    "science" : 234
  }
}
{
  "_id" : ObjectId("60227eaff19652db63812e8f"),
  "name" : "Chandhan",
  "age" : 18
}
> ]
```

- **findOne ():** The findOne () method returns a single document object, or null if no document is found. You can pass a query object to this method to filter the results.

Syntax:

```
db.collectionName.findOne()
```

Example:

```
db.users.findOne({ name: "Jim" })
```

- **Output:** - { "id": ObjectId("6425546aabfa5ebad9b5e2fa"), "name" : "Jim", "age" : 29 }

- **Update:** In MongoDB, the "update" operation is used to modify existing documents in a collection. You can perform update operations using the following methods provided by the MongoDB:

Method	Description
<code>db.collection.updateOne()</code>	It is used to update a single document in the collection that satisfy the given criteria.
<code>db.collection.updateMany()</code>	It is used to update multiple documents in the collection that satisfy the given criteria.

- **Update One ():**

Syntax:

```
db.collectionName.updateOne(filter, update, options)
```

- **Example:** `db.users.updateOne({ name: "Angela" }, { $set: { email: "angela@gmail.com" } })`
- **Output:** `{ "_id" : ObjectId("64255335abfa5ebad9b5e2f7"), "name" : "Angela", "age" : 27, "email" : "angela@gmail.com" }`

- **Update Many ():**

Syntax:

```
db.collectionName.updateMany(filter, update, options)
```

Example:

```
db.users.updateMany({ age: { $lt: 30 } }, { $set: { status: "active" } })
```

- **Output:** `{ "_id" : ObjectId("64255335abfa5ebad9b5e2f7"), "name" : "Angela", "age" : 27, "email" : "angela@gmail.com", "status" : "active" }`

- { "_id" : ObjectId("6425546aabfa5ebad9b5e2f8"), "name" : "Angela", "age" : 27, "status" : "active" }
- { "_id" : ObjectId("6425546aabfa5ebad9b5e2f9"), "name" : "Dwight", "age" : 30 }
- { "_id" : ObjectId("6425546aabfa5ebad9b5e2fa"), "name" : "Jim", "age" : 29, "status" : "active" }

➤ **Delete:** In MongoDB, the "delete" operation is used to remove documents from a collection. There are several ways to perform a delete operation, including the following:

- **Delete One ():** The deleteOne() method is used to remove a single document that matches a specified filter.

Syntax:

```
db.collectionName.deleteOne(filter, options)
```

filter: Specifies deletion criteria using query operators. Specify an empty document {} to delete the first document returned in the collection.

Example:

```
db.users.deleteOne({ name: "Angela" })
```

- **Delete Many ():** The deleteMany() method is used to remove multiple documents that match a specified filter.

Syntax:

```
db.collectionName.deleteMany(filter, options)
```

Example:

```
db.users.deleteMany({ age: { $lt: 30 } })
```

- **Drop:** The drop() method is used to remove an entire collection.

Syntax:

```
db.collectionName.drop()
```

Example:

```
db.users.drop()
```

Output:

```
true
```

❖ Indexing:

- Indexing in MongoDB is a crucial feature that enhances query processing efficiency. Without indexing, MongoDB must scan every document in a collection to retrieve the **matching** documents and leading to **slower query performance**.
- Indexes are special data structures that store some information related to the documents such that it becomes easy for MongoDB to find the right data file. The indexes are ordered by the value of the field specified in the index.
- **Creating an Index:** MongoDB provides a method called **createIndex()** that allows users to create an index.

```
db.COLLECTION_NAME.createIndex({KEY:1})
```

The key determines the field on the basis of which we want to create an index and 1 (or -1) determines the order in which these indexes will be arranged(ascending or descending).

Example of Indexing in MongoDB

```
db.mycol.createIndex({"age":1})
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 1,
  "numIndexesAfter" : 2,
  "ok" : 1
}
```

- **Drop Index:** In order to drop an index, MongoDB provides the **dropIndex()** method.

Syntax:

```
db.NAME_OF_COLLECTION.dropIndex({KEY:1})
```

The dropIndex() methods can only delete one index at a time. In order to delete (or drop) multiple indexes from the collection, MongoDB provides the dropIndexes() method that takes multiple indexes as its parameters.

Syntax:

```
db.NAME_OF_COLLECTION.dropIndexes({KEY1:1, KEY2:1})
```

- **Get Index:** The **getIndexes()** method in MongoDB gives a description of all the indexes that exists in the given collection.

Syntax

```
db.NAME_OF_COLLECTION.getIndexes()
```

It will retrieve all the description of the indexes created within the collection.

❖ Aggregation:

- Aggregation in MongoDB is a powerful feature that enables developers to perform complex data transformations and computations on collections of documents.
- It enables us to group, filter, and manipulate data to produce summarized results.
- MongoDB uses an aggregation pipeline, which is a sequence of steps that processes data. Each step transforms the data as it moves through the pipeline.
- The pipeline lets you filter, group, sort, reshape, and calculate data as it flows through each step.
- **Following is the various common pipeline functions used in MongoDB:**
 1. **\$group:** Groups documents by a specified identifier and performs aggregation. It is similar to GROUP BY in SQL.

```
db.users.aggregate([
  { $group: { _id: "$city", averageAge: { $avg: "$age" } } }
])
```

It **Groups** documents by the city field and calculates the average age using the **\$avg** accumulator.

2. **\$match:** Filters documents based on conditions. It is similar to WHERE clause in SQL.

• Example:

javascript

```
db.orders.aggregate([
  { $match: { status: "Shipped" } }
]);
```

This filters orders where the `status` is `"Shipped"`.

3. **\$sort:** Sorts documents based on a specified field. It is similar to the ORDER BY clause in SQL.

- **Example:**

javascript

```
db.orders.aggregate([
  { $sort: { orderDate: -1 } }
]);
```

This sorts the documents in descending order of `orderDate`.

4. **\$project:** Selects or reshapes specific fields from the documents. It Include or exclude fields from the output documents.

```
db.users.aggregate([
  { $project: { name: 1, city: 1, _id: 0 } }
])
```

Output:

```
[
  { name: 'Alice', city: 'New York' },
  { name: 'Bob', city: 'Los Angeles' },
  { name: 'Charlie', city: 'Chicago' }
]
```

5. **\$limit:** Limit the number of documents passed to the next stage.

- **Example:**

javascript

```
db.orders.aggregate([
  { $limit: 5 }
]);
```

This will return the first 5 documents from the collection.

6. **\$skip**: Skips a specified number of documents.

- **Example:**

```
javascript

db.orders.aggregate([
  { $skip: 10 }
]);
```

This will skip the first 10 documents and return the rest.

7. **\$count**: Counts the number of documents in the pipeline.

- **Example:**

```
javascript

db.orders.aggregate([
  { $match: { status: "Shipped" } },
  { $count: "shippedOrders" }
]);
```

This counts the number of documents where the `status` is `"Shipped"` and outputs a document with the field `shippedOrders` holding the count.

❖ **MapReduce:**

- MongoDB Map-Reduce is a data processing model that facilitates operations on large data sets to produce aggregated results. It uses the **mapReduce** () function comprising **map** and **reduce** functions to handle complex data transformations.
- The map-reduce function first queries the collection, then maps the result documents to emit key-value pairs, which is then reduced based on the keys that have multiple values.

- Example:

Input Document:

```
{
  "_id": 1,
  "name": "Shivam Singla",
  "subject": "maths",
  "score": 80
},
{
  "_id": 2,
  "name": "Sachin Singla",
  "subject": "english",
  "score": 90
},
{
  "_id": 3,
  "name": "Shivam Gupta",
  "subject": "maths",
  "score": 95
}
```

MongoDB MapReduce operation to calculate the total score:

```
db.scores.mapReduce(
  function() {
    emit("total_score", this.score);
  },
  function(key, values) {
    var totalScore = 0;
    values.forEach(function(value) {
      totalScore += value;
    });
    return { value: totalScore };
  },
  { out: { inline: 1 } }
)
```

Output:

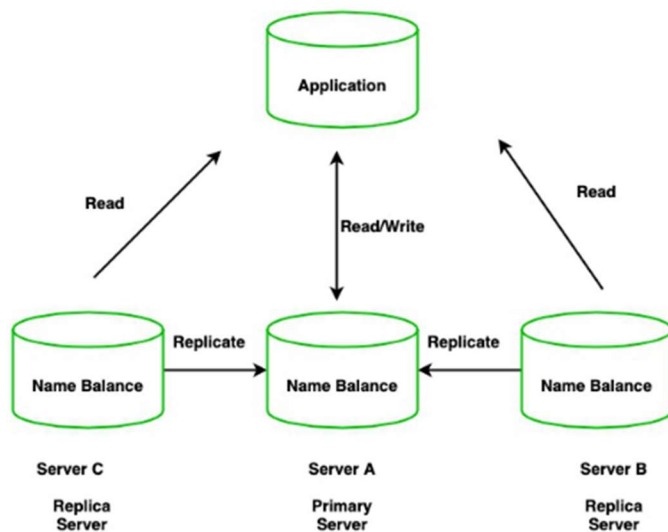
```
{
  "_id": "total",
  "value": 245
}
```

Explanation:

1. The **map** function emits a key-value pair for each document in the **scores** collection. The key is set to a constant string "total" and the value is set to the **score** field of the document.
2. The **reduce** function aggregates the values for each key by summing them up.
3. The **out** option specifies the output collection for the MongoDB MapReduce result. In this case, we use the **{ inline: 1 }** option to return the result as an array of documents rather than storing it in a collection.
4. The result is a single document with a key **_id** of "total" and a value of 245, which is the sum of the **score** fields in the input documents.

❖ **Replication:**

- Replication is the method of duplication of data across multiple servers in MongoDB.
- **For example**, we have an application that reads and writes data to a database and says server A has a name and balance which will be copied/replicated to two other servers in two different locations.



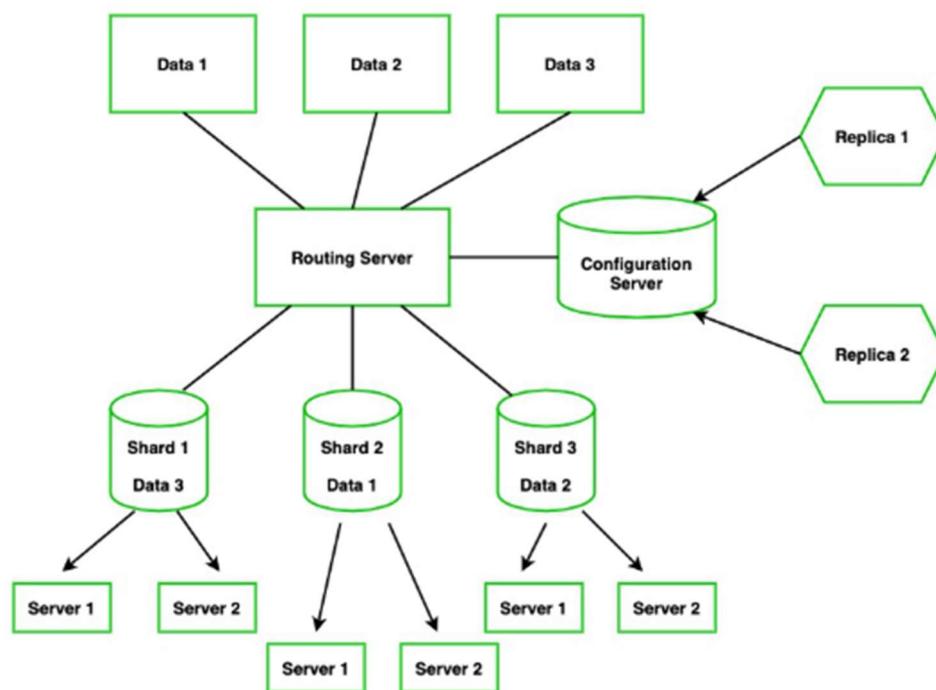
- Replication increases redundancy and data availability with multiple copies of data on different database servers. So, it will increase the performance of reading scaling. The set of servers that maintain the same copy of data is known as **replica servers** or **MongoDB instances**.

❖ **Sharding:**

- In MongoDB, sharding is useful when no single server can handle large, modern-day workloads. Due to digitization, there is an ever-growing demand to capture data, which poses increased load on the database.
- However, all the resources (memory, disk, CPU) have a saturation point, after which they cannot take any more requests, and the system response time will increase, thus impacting system stability and performance.
- **Sharding** involves distributing data across numerous machines. MongoDB employs sharding to handle deployments involving extensive data volumes and high-performance operations.
- It allows you to scale the entire database horizontally. Horizontal scaling, also known as range partitioning, this technique splits data into rows across different servers. Each server, or shard, contains a subset of the rows, but all columns from the original table are included.
- A shard is a replica set that contains a subset of the cluster's data.

• **Working:**

- Sharding determines the problem with horizontal scaling.
- It breaks the system dataset and store it over multiple servers, adding new servers to increase the volume as needed.



- Now, instead of one server as primary, we have multiple servers called **Shard**. We have different routing servers that will route data to the shard servers.
 - **For example:** Let say we have **Data 1, Data 2, and Data 3** this will be going to the routing server which will route the data (i.e., Different Data will go to a particular Shard). Each Shard holds some pieces of data.
 - Here the **configuration server** will hold the metadata and it will configure the routing server to integrate the particular data to a shard however configuration server is the MongoDB instance if it goes down then the entire server will go down, so it again has **Replica Configure database**.
-